

**CENTRO EDUCACIONAL DA FUNDAÇÃO SALVADOR ARENA
FACULDADE ENGENHEIRO SALVADOR ARENA**

GUSTAVO TRINDADE RODRIGUES

LUCAS BARBOZA SILVA

MATHEUS HENRIQUE GONÇALVES NUNES

VICTOR OLIVEIRA MALVÃO

**PROJETO PARA COMPOSIÇÃO DE NOTA: GERENCIADOR DE
PROCESSOS**

**São Bernardo do Campo
2026**

GUSTAVO TRINDADE RODRIGUES

LUCAS BARBOZA SILVA

MATHEUS HENRIQUE GONÇALVES NUNES

VICTOR OLIVEIRA MALVÃO

**PROJETO PARA COMPOSIÇÃO DE NOTA: GERENCIADOR DE
PROCESSOS**

Projeto apresentado para composição de nota em Sistemas Operacionais do curso de Engenharia da Computação da Faculdade Engenheiro Salvador Arena, sob a orientação do Prof. Dr. Vinicius da Silva Borges.

**São Bernardo do Campo
2026**

SUMÁRIO

1	Visão geral	3
2	Separação entre política e mecanismo	4
3	Diagrama de módulos	5
4	Estrutura de uma tarefa	6
5	Interface dos módulos	9
6	Parâmetros configuráveis	12
7	Funcionamento interno	13
8	Convenções de simulação	14
9	Dependências e ambiente	15

1 VISÃO GERAL

O simulador reproduz o funcionamento de um escalonador de tarefas em um único processador. A partir de um conjunto de tarefas definido pelo usuário (cada uma com instante de ingresso, tempo de processamento e prioridade), o programa executa, tick a tick, um dos seis modelos de escalonamento disponíveis:

- First-Come First-Served;
- Shortest Job First;
- Shortest Remaining Time First;
- Round-Robin;
- Prioridade Cooperativa; e
- Prioridade Preemptiva.

O resultado é apresentado como um diagrama de tempo, no qual cada tarefa ocupa uma raia horizontal que mostra, tick a tick, se ela está em execução, em espera, suspensa aguardando um recurso, ou já concluída.

O simulador também reproduz a disputa por um recurso de uso exclusivo R , permitindo observar suspensão de tarefas e trocas de contexto.

A interface principal *TelaPrincipal* permite simular um único escalonador de cada vez ou comparar os seis simultaneamente sobre o mesmo conjunto de tarefas, além de manter um painel de histórico textual dos eventos da simulação.

2 SEPARAÇÃO ENTRE POLÍTICA E MECANISMO

O projeto concentra o mecanismo de simulação na classe *TabelaGrafico*, especificamente nas funções *tick()* e *mecanismo_geral()*. O mecanismo é único e compartilhado por todos os algoritmos: a cada chamada de *tick()*, o tempo global *tempo_atual* avança uma unidade, o display é atualizado e o laço se rechama após o intervalo configurado, até que todas as tarefas estejam finalizadas.

A política (decisão de qual tarefa deve ocupar o processador no tick atual) fica isolada em seis funções específicas, uma por algoritmo:

- *processo_first_come()*;
- *processo_shortest_job()*;
- *processo_shortest_remaining_time()*;
- *processo_round_robin()*;
- *processo_cooperativa()*; e
- *processo_preemptiva()*.

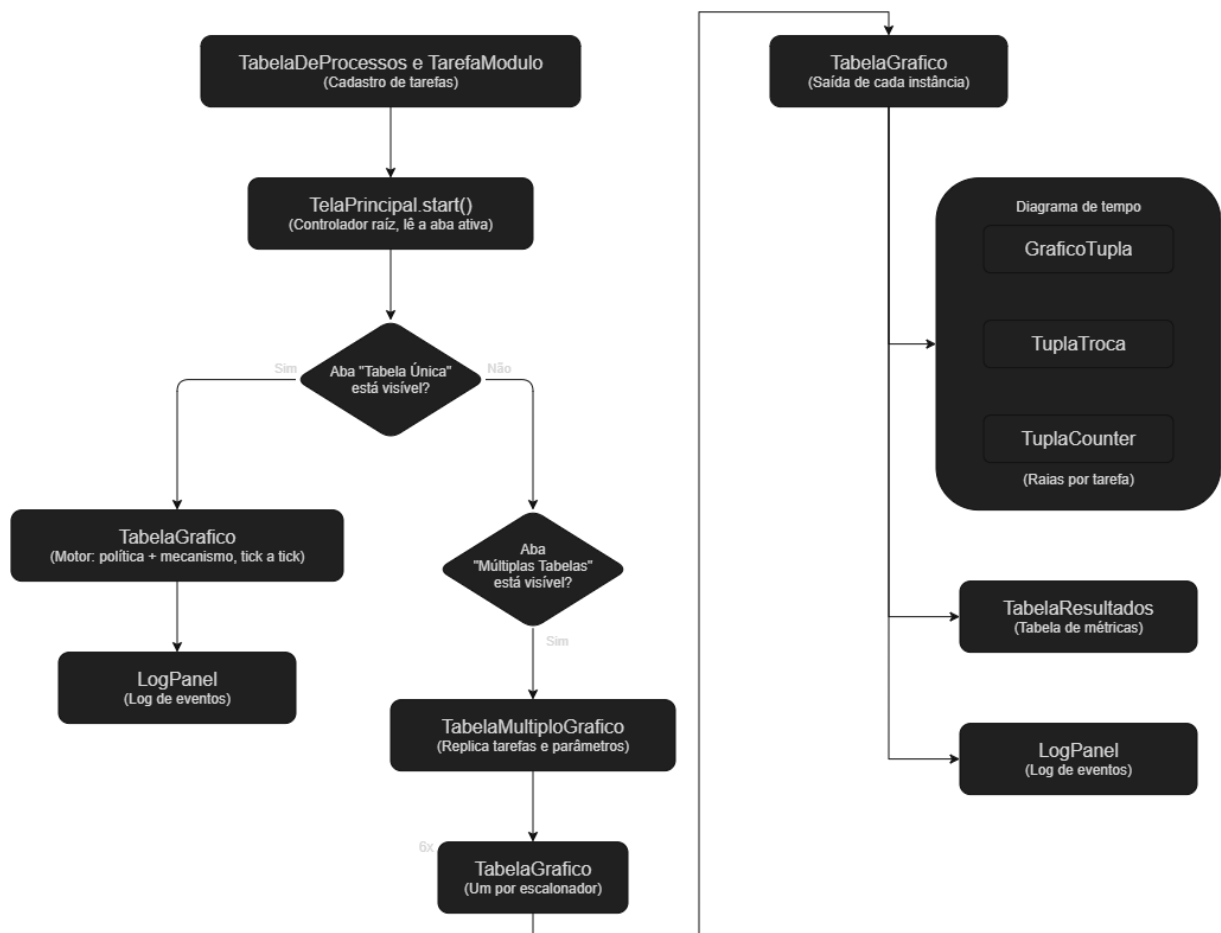
Cada uma delas recebe o mesmo conjunto de tarefas prontas e devolve apenas qual *TarefaInfo* deve ser executada (ou *null*, se nenhuma estiver pronta). Nenhuma dessas funções manipula diretamente o relógio, o diagrama de tempo ou o log; essa responsabilidade é exclusiva do mecanismo.

A relação entre as duas camadas é a função *mecanismo_geral(tarefaAtiva)*, que recebe a tarefa escolhida pela política e, sem conhecer o critério que a elegeu, decide se uma troca de contexto deve ser cobrada, avança a raia da tarefa ativa no diagrama, registra o instante da primeira execução e envia mensagens ao log. Esse desenho corresponde exatamente à separação clássica entre mecanismo (comum, agnóstico ao algoritmo) e política (específica de cada algoritmo). O tratamento do recurso compartilhado, feito por *gerenciarRecurso()*, é chamado individualmente dentro de cada função de política, e não centralizado no mecanismo, gerando repetição da mesma chamada nas seis políticas.

3 DIAGRAMA DE MÓDULOS

O fluxo de dados pode ser interpretado da seguinte maneira: o módulo de entrada de tarefas *TabelaDeProcessos* produz uma lista de *TarefaInfo*; *TelaPrincipal* entrega essa lista ao motor de simulação (uma única instância de *TabelaGrafico* ou seis instâncias coordenadas por *TabelaMultiploGrafico*); o motor aplica política e mecanismo tick a tick e alimenta duas saídas visuais: o diagrama de tempo (raias *GraficoTupla*, *TuplaTroca*, *TuplaCounter*) e o painel de log (*LogPanel*, compartilhado por ambos os modos). O fluxo pode ser visualizado na figura 1.

Figura 1: Diagrama de módulos



Fonte: Própria

4 ESTRUTURA DE UMA TAREFA

Os campos do recurso *TabelaInfo* pode ser visualizado na tabela 1.

Tabela 1: Estrutura da tarefa

Campo	Tipo	Função
<i>nome</i>	<i>string</i>	Nome de exibição da tarefa
<i>uid</i>	inteiro (padrão 0)	Identificador único da tarefa
<i>Tempo_Processo</i>	inteiro, 1 a 999 (padrão 2)	Tempo total de processamento (<i>tp</i>) demandado pela tarefa
<i>Tempo_Entrada</i>	inteiro, 1 a 999 (padrão 0)	Instante (<i>tick</i>) em que a tarefa ingressa no sistema
<i>Prioridade</i>	inteiro, 0 a 999 (padrão 1)	Prioridade estática da tarefa; maior valor indica prioridade mais alta
<i>TempoPedidoRecurso</i>	inteiro, -1 a 999 (padrão 0)	<i>Ticks</i> de execução própria após os quais a tarefa solicita o recurso <i>R</i>
<i>TempoUsoRecurso</i>	inteiro, 1 a 999 (padrão 0)	<i>Ticks</i> que a tarefa mantém o recurso <i>R</i> após obtê-lo; o valor padrão 0 indica, na prática, que a tarefa não disputa o recurso
<i>Tempo_Execucao</i>	inteiro (padrão 0)	<i>Turnaround</i> : <i>tick</i> em que a tarefa terminou, medido a partir do próprio ingresso
<i>Tempo_Espera</i>	inteiro (padrão 0), calculado	Tempo de espera, calculado por <i>CalcularTempoEspera()</i>
<i>Tempo_Primeira_Execucao</i>	inteiro (padrão 0)	Tempo de resposta: <i>tick</i> em que a tarefa foi escolhida pela primeira vez, a partir do ingresso
<i>Executou_Antes</i>	<i>booleano</i> (padrão falso)	Indica se a tarefa já foi escolhida para execução ao menos uma vez
<i>PediuRecurso</i>	<i>booleano</i> (padrão falso)	Indica se a tarefa já sinalizou o pedido do recurso <i>R</i>

5 INTERFACE DOS MÓDULOS

TabelaDeProcessos: controla o cadastro de tarefas na tela.

- *criar_tarefa(prioridade, tempo_entrada, tempo_processo, tempo_pedido_recurso, tempo_uso_recurso)*: recebe os cinco parâmetros de uma tarefa; não devolve valor; cria uma nova *TarefaInfo* e instancia seu elemento visual na lista.
- *criar_array_aleatorio()*: sem parâmetros; sem retorno; gera de 1 a 8 tarefas com valores aleatórios.
- *get_all_tarefas()*: sem parâmetros; devolve *Array[TarefaInfo]*; extrai a tarefa de cada elemento visual da lista.
- *save() / load_dict() / set_valores_from_dict()*: gravam e recuperam o conjunto de tarefas em arquivos *.save (JSON)* em *user://*.

TarefaModulo: cartão visual de uma tarefa na lista de cadastro.

- *_ready()*: sorteia um nome caso a tarefa não tenha um, preenche a descrição textual e inicia a animação de entrada.
- *atualizar_titulo()*: atualiza o texto exibido com nome e UID.
- *get_selecionado()*: devolve um booleano indicando se o cartão está marcado para remoção.

TarefaInfo (extends Resource): estrutura de dados de uma tarefa.

- *CalcularTempoEspera()*: sem parâmetros; sem retorno; grava em *Tempo_Espera* o resultado de *Tempo_Execucao - Tempo_Processo*.

TabelaGrafico: motor de simulação e diagrama de tempo de um único escalonador.

- *start(tarefas: Array[TarefaInfo])*: recebe o conjunto de tarefas; sem retorno; zera contadores, monta as raias (uma por tarefa, mais a de troca de contexto e a régua de tempo) e inicia *tick()*.

- *tick(processo_id)*: recebe um identificador para descartar chamadas obsoletas; sem retorno; despacha a política do escalonador selecionado e avança *tempo_atual*.
- *processo_first_come()*, *processo_shortest_job()*,
processo_shortest_remaining_time(), *processo_round_robin()*,
processo_cooperativa(), *processo_preemptiva()*: sem parâmetros explícitos; cada uma chama *mecanismo_geral()* com a *TarefaInfo* escolhida (ou *null*).
- *mecanismo_geral(tarefaAtiva)*: recebe a tarefa escolhida; sem retorno; decide a cobrança de troca de contexto, avança a raia ativa e registra o *log*.
- *gerenciarRecurso(tupla)*: recebe a raia candidata; sem retorno; sinaliza o pedido do recurso *R*.
- *liberarRecurso()* (estática): sem parâmetros; sem retorno; zera o detentor do recurso *R*.

GraficoTupla: raia individual de uma tarefa no diagrama de tempo.

- *step(tempo_atual, ativo)*: recebe o *tick* e se a tarefa foi escolhida; sem retorno; desenha o estado da raia nesse *tick* e, ao concluir a tarefa, grava *Tempo_Execucao* e chama *CalcularTempoEspera()*.
- *get_finalizado()*: devolve *booleano*; verdadeiro quando os *ticks* ativos atingem *Tempo_Processo*.
- *estaSuspendido()*: devolve *booleano*; verdadeiro quando o recurso *R* pertence a outra tarefa e ela acabou de atingir seu *tick* de solicitação.

TuplaCounter (herda *GraficoTupla*): régua de tempo no topo do diagrama.

TuplaTroca (herda *GraficoTupla*): raia de troca de contexto.

- *get_finalizado()*: verdadeiro quando os ticks de troca cobrem o custo configurado.
- *step()*: preenche a raia enquanto o custo não é totalmente cobrado.
- *reiniciar()*: zera o contador de ticks da troca.

TabelaResultados (extends *GridContainer*): monta a tabela de métricas na tela.

- *add_tuple(nome, tempo_execucao, tempo_processo, tempo_espera, primeira_execucao, total)*: recebe os seis valores de uma linha; sem retorno; cria o cabeçalho na primeira chamada e adiciona a linha.
- *clear()*: sem parâmetros; sem retorno; remove todas as linhas.

TabelaMultiploGrafico: compara os seis escalonadores (ou cenários salvos) simultaneamente.

- *start(tarefas)*: replica os parâmetros configurados para uma *TabelaGrafico* por escalonador e dispara todas.
- *get_all_scenarios() / dict_to_tarefas()*: recuperam cenários salvos e os convertem em *Array[TarefaInfo]*.

LogPanel: histórico textual, compartilhado por todos os modos.

- *clear()*: limpa o texto.
- *queue_message(msg)*: anexa uma mensagem ao final do texto.

TelaPrincipal: controlador raiz.

- *start()*: lê as tarefas cadastradas e as envia para o motor único ou para o comparador, conforme a aba ativa.

EntradaDeTexto: campo numérico reutilizável.

- *get_entrada()*: devolve um *float* com o valor do campo.
- *set_enabled(n)*: habilita ou desabilita a edição.

6 PARÂMETROS CONFIGURÁVEIS

A lista de parâmetros configuráveis pode ser visualizada na tabela 1.

Tabela 1: Parâmetros configuráveis

Parâmetro	Campo na interface	Faixa válida	Valor padrão
Quantum (tq)	"Tempo de quantum"	Numérico inteiro; a interface recusa valores ≤ 0 .	1
Custo da troca de contexto (ttc)	"Custo de troca de contexto"	Numérico inteiro; a interface recusa valores ≤ 0 .	1
Passo do envelhecimento (α)	"Taxa de envelhecimento"	Numérico inteiro; a interface recusa valores ≤ 0 .	1
Mecanismo de correção (herança/teto)	Botões "Herança" e "Teto"	Não numérico; dois botões visíveis apenas na Prioridade Preemptiva	Não aplicável

7 FUNCIONAMENTO INTERNO

A cada chamada de *tick()*, o motor executa a função de política do escalonador selecionado e avança *tempo_atual* em uma unidade.

- **Montagem do conjunto de tarefas prontas:** Cada política filtra as raiaas ativas mantendo as tarefas cujo *Tempo_Entrada* já foi alcançado, que ainda não finalizaram e que não estão suspensas aguardando o recurso R. Sobre esse conjunto, cada política aplica seu critério: uid crescente no FCFS e no Round-Robin; menor *Tempo_Processo* no SJF; menor tempo restante no SRTF; maior Prioridade na cooperativa.
- **Cobrança da troca de contexto:** Ocorre dentro de *mecanismo_geral()*: quando a tarefa escolhida difere da última que efetivamente executou e o custo configurado é maior que zero, a raia de troca consome ticks antes que a nova tarefa volte a avançar seus próprios ticks. O custo é cobrado antes da execução da tarefa recém-selecionada e soma-se ao tempo total decorrido, não é descontado da fatia de execução.
- **Suspensão por disputa do recurso:** Uma tarefa sinaliza o pedido do recurso quando seus próprios ticks de execução atingem *TempoPedidoRecurso*. Enquanto o recurso estiver em posse de outra tarefa, a tarefa solicitante é excluída do conjunto de prontas em todas as políticas, permanecendo suspensa até a liberação do recurso (quando *ticks_recurso* atinge *TempoUsoRecurso*).
- **Cálculo das métricas ao final de cada tarefa:** Quando uma tarefa conclui seus ticks de execução, *GraficoTupla.step()* grava *Tempo_Execucao* (turnaround, relativo ao ingresso) e chama *CalcularTempoEspera()*, que grava $\text{Tempo_Espera} = \text{Tempo_Execucao} - \text{Tempo_Processo}$, a fórmula clássica de tempo de espera a partir do turnaround e do tempo de processamento.

8 CONVENÇÕES DE SIMULAÇÃO

- **Unidade de tempo:** um tick corresponde a uma chamada de tick(); todos os tempos são medidos nessa unidade.
- **Critério de desempate:** no FCFS e no Round-Robin, tarefas empatadas são ordenadas por uid crescente (ordem de criação). No SJF e na prioridade cooperativa não há critério de desempate explícito além da ordem interna da lista de raias ativas.
- **Momento da cobrança da troca de contexto:** antes da execução da tarefa recém-escolhida; o custo é somado ao tempo total, não descontado da fatia de execução.
- **Posição de retorno após preempção:** aplicável apenas ao Round-Robin; a tarefa preemptada retorna ao final da fila.
- **Sentido da escala de prioridades:** valores maiores de Prioridade indicam prioridade mais alta e são escolhidos primeiro.
- **Referência de tempo para o uso do recurso:** TempoPedidoRecurso e TempoUsoRecurso são contados a partir dos ticks de execução próprios da tarefa, não do tempo global do sistema.
- **Cálculo do tempo de espera:** turnaround (Tempo_Execucao, já relativo ao ingresso) menos tempo de processamento (Tempo_Processo).

9 DEPENDÊNCIAS E AMBIENTE

- Godot Engine 4.4 ou superior.